Filter provenance and predicted-response strategy

Filter provenance and predicted-response strategy

This document defines how a room-correction filter becomes a deployable, auditable bundle and how `omdrcctrl` can display the measured and predicted responses of the filter that BruteFIR is actually using.

The central rule is:

> A graph is selected by the hashes of the coefficient files loaded by the running BruteFIR process, never only by a geometry name, sample rate, filename, or UI selection.

Hashes prove byte identity and protect an established relationship. They cannot retroactively prove the human design decision that a filter was made from a particular measurement. That relationship must be declared when the source bundle is exported, checked where possible, and committed with the artifacts.

Audit of `120.blue` on 2026-08-13

The adjacent source repository was `../DRC/DRC-120.blue` at commit `d7a8e4dbf9418e43e2397bb140a8f81edcab1087`. The selected base source files are tracked and unmodified. The source repository also contains unrelated untracked files, so its complete worktree is not clean.

Update on 2026-08-14: the adjacent checkout now contains a newer, uncommitted candidate. In particular the current FLX/FRX trimmed WAVs and many files under `new.filters.txts` are modified, and no annotated tag names this candidate yet. The concrete command later in this document is therefore a valid role-selection *dry run*, not authorization to deploy those working-tree bytes. Commit the chosen artifacts and generated declaration together and create the annotated tag before running `new_filter_design.py`.

Base filter: verified deployment lineage

The source WAVs copied into this repository are byte-identical:

Channel	Source and repository copy	SHA-256
Left	`FLX-trimmed-48k.wav`	`4116b567012c0fb857b9a7f1a5a70e5dca16188a`
Right	`FRX-trimmed-48k.wav`	`40a28cc294e94532b17edd5ab24a867c9a7cefb7`

Both are mono, 48,000 Hz, signed 32-bit PCM and contain 131,072 samples. `filters/120.blue/48000/L.raw` and `R.raw` are exact, sample-for-sample float64 conversions of those WAV samples.

All five committed rate pairs were also regenerated in a temporary directory with the current `scripts/REW2raw-all-rates.sh`. Every regenerated RAW file was byte-identical to the committed file:

Rate	Left SHA-256	Right SHA-256
44,100	`afcc63a4d856008b9c68739de60b`	
48,000	`e0881d86077f25413c1c010968b0`	
88,200	`f5d7775909f6023708ba3c9ae00b`	
96,000	`986a7d4f5a72402c89a29ba1e32e`	
192,000	`d5eda1b8aca4247e74886424d1bc`	

The five base BruteFIR templates select the matching rate directory, `L.raw` and `R.raw`, and declare `FLOAT64_LE`. Therefore the chain from the committed base 48 kHz WAVs to every deployed base RAW is reproducible and verified.

Filter TXT-to-WAV correspondence

The filter response exports have these hashes:

File	SHA-256
`txt/FLX-trimmed.txt`	`69bfd876173f7bb5c39876d547da02d4132d819eb27aacc25cc6985e`
`txt/FRX-trimmed.txt`	`da255d770d0c2b39257d3f14a6f1c9d8ecd4dd11d87b76159e50d8a0`

Each TXT has 65,533 response rows. Its frequency values map exactly, within the printed decimal precision, to FFT bins 3 through 65,535 of its corresponding 131,072-sample WAV. The WAV has a 24,000-sample (500 ms) causal export delay. After removing exactly that delay, the WAV FFT agrees closely with the exported magnitude and phase. Across the complete export the RMS errors are 0.0125 dB and 0.0946 degrees for FLX, and 0.0018 dB and 0.0119 degrees for FRX. Above 100 Hz, the worst errors are 0.049 dB/0.454 degrees for FLX and 0.040 dB/0.400 degrees for FRX.

The largest FLX differences are around 27–31 Hz (up to 1.116 dB and 13.28 degrees). They are consistent with exporting a finite, windowed causal impulse from a frequency-domain response; consequently the TXT and WAV are not expected to be two byte-equivalent encodings of identical numbers. A future export should also include a response TXT made by re-importing the final WAV into REW. That second TXT provides an independent, post-window reference that can be compared much more tightly to the Python FFT.

Historical base-bundle REW audit (not a deployment dependency)

The earlier audit of the already deployed default bundle used `120-blue-with-inversion.mdat`, SHA-256 `b184b824236868a898c33877a56d0f1003a2e442922d8b4f9c05ef1a51b8d6c7`. It was opened with REW V5.40 Beta 132 using `-nogui -noaudio -api`; no audio device was opened. All TXT exports resolve to unique traces in its 44-trace inventory.

The original `L.txt` and `R.txt` hypothesis is wrong: those are older January 2025 measurements. The actual source pair and independent sum were measured with the same setup and acoustic reference within 109 seconds:

Role	Trace / UUID	TXT SHA-256
Left	`L.120.Blue` / `51ba95b5-1b8e-4303-8c8d-f1828cd75463`	`aa829ea8d0ef4ae97802f9c88fe7e53688c0b694f`
Right	`R.120.Blue` / `1f7ef644-083c-47c2-b65f-e116b89f3d50`	`56e11fdb76bcc0f5d729fed3eb446fa6c2af3c29b`
Independent sum	`L+R.120.Blue` / `d0bbd016-69e2-43a8-9c92-822618602af7`	`6555b6bfbad78c93b315b47861d8940b55feb47d3`

The project notes preserve the arithmetic chain: L/R are multiplied by `X801 (revised)`, converted to minimum phase, inverted against the RMS/phase-average target with 0 dB maximum gain over 20–225 Hz, converted to `LFilter`/`RFilter`, multiplied by `X801`, and trimmed to `FLX-trimmed`/`FRX-trimmed`. The final trace UUIDs are `c2515dc4-51d7-4e96-9d3a-2be205738808` and `7815c6db-1e74-4957-8573-f0fa419e2b32`.

REW's API impulse arrays for those final traces match the source WAVs within float32-percent to PCM-S32 quantisation: maximum absolute errors are `1.43e-8` left and `2.15e-8` right. This closes the semantic gap from measured traces, through the recorded REW operations, to the deployed source WAVs.

This paragraph is historical evidence for that base bundle, not part of the new declaration/tag procedure. The optional project corresponding to the current Rscreen work is `120.blue.Rscreen.mdat`. The current example deliberately omits it: neither declaration, deployment nor plotting opens an `.mdat`, and no project hash is required when the exported artifacts, declaration and annotated tag are the chosen trust boundary.

Known failures that must not receive a green status

1. The base pair needs at most 2.3 dB attenuation (including the 1 dB safety margin); all base configurations specify 3.0 dB and pass. The repaired `scripts/headroom_calc.py filters/120.blue` discovers the real rate pairs.

2. The committed `+2dB` 192 kHz RAW pair is not byte-reproducible from the two currently committed `filters/120.blue/rew/+2dB` WAVs. Apart from tiny sample differences, each channel has a constant gain discrepancy: approximately -0.12996 dB left and -0.11876 dB right. The adjacent DRC repository has a tracked left +2 dB WAV but no matching tracked right WAV, and the left file uses a different WAV encoding. This variant needs to be re-exported and redeployed as a new bundle; until then it is **unverified**.
3. No manifest is intentionally supplied for `+2dB`. The web UI therefore reports it as unverified and withholds stored measurements, while retaining only a live active-filter diagnostic.

Provenance bundle

Each immutable design inside a geometry needs one manifest. The default design keeps the legacy paths; additional A/B designs use an `@design-id` selector:

```
filters/<geometry>/
  provenance/
    default.json
    2026-08-target-a.json
  source/
    2026-08-target-a/
      measurement-L.txt
      measurement-R.txt
      measurement-L+R.txt
      filter-L.txt
      filter-R.txt
      filter-L.wav
      filter-R.wav
      corrected-L+R-independent.txt    # optional numerical cross-check
  analysis/
    2026-08-target-a.json
  44100/L.raw
  44100/R.raw
  44100/@2026-08-target-a/L.raw
  44100/@2026-08-target-a/R.raw
  ...
configs/<geometry>/
  brutefir-44100.conf.in
  brutefir-44100@2026-08-target-a.conf.in
```

Source filenames inside a bundle use stable logical roles. The original REW names remain metadata in the manifest. Copying the selected sources into this repository is intentional: a deployment must not depend on a mutable sibling checkout or an absolute path that will not exist on the installed machine.

The manifest should contain at least:

- schema version, geometry, variant, creation time, and a content-derived `bundle_id`;
- source repository URL, exact Git commit, annotated tag name and immutable tag object ID, plus the source declaration's path, Git blob and SHA-256;
- optional `.mdat` path/SHA-256 as archival evidence, and designer-declared trace lineage; neither deployment nor plotting opens the project;
- for every source, RAW, config template, and analysis file: logical role, relative path, byte size, media/sample format, sample rate/count where applicable, and SHA-256;
- parsed REW header metadata: measurement name/date, source, notes, smoothing, frequency step, timing-reference/delay information, and REW version;
- derivation commands and versions of the deployment script, Python, NumPy, SoX, and the resampling flags;
- TXT/WAV validation results, independent REW predicted-response comparison results, headroom per channel, safety margin, and required attenuation;
- the exact runtime rate/variant mapping and the expected BruteFIR format and attenuation.

The `bundle_id` is the SHA-256 of a canonical identity containing a hash of the complete source-provenance object (repository, commit, annotated tag object, declaration, lineage and attestation), all source artifact

hashes, runtime config/RAW hashes and settings, and the analysis hash. Thus editing any provenance shown in the UI invalidates the bundle instead of changing a cosmetic label. An annotated tag establishes an immutable named Git object; a signed annotated tag additionally establishes authorship.

Deployment command and transaction

For a new design, first create the role declaration in the source repository. The declaration command never invokes REW. A read-only discovery mode can use the newest `.mdat` filename solely to find its sibling `<stem>.txts` directory and print a candidate command:

```
python3 scripts/declare_filter_design.py \
  --suggest-from-source-root ../DRC/DRC-120.blue
```

Newest means filesystem modification time, not Git history or semantic authority. The `.mdat` is never opened or added to the suggested command. The tool selects and reports compatible L/R alternatives from `<stem>.txts`, then searches sibling `*.txts` directories and root WAVs for the aggregate, filters and optional corrected exports. The result remains only a suggestion; the printed declaration command performs the actual header, grid, hash, TXT/WAV and convolution checks.

For an explicit declaration, run:

```
python3 scripts/declare_filter_design.py \
  --source-root ../DRC/DRC-120.blue \
  --geometry 120.blue --design-id rscreen-fdw8-20260813 \
  --description "120.blue Rscreen, 8-cycle FDW correction" \
  --measurement-left "new.filters.txts/L 120.Rscreen.orig.txt" \
  --measurement-right "new.filters.txts/R 120.Rscreen.orig.txt" \
  --measurement-sum new.filters.txts/LR.orig.txt \
  --filter-left-txt new.filters.txts/FLX.txt \
  --filter-right-txt new.filters.txts/FRX.txt \
  --filter-left-wav FLX-trimmed-48k.wav \
  --filter-right-wav FRX-trimmed-48k.wav \
  --corrected-left-txt new.filters.txts/L.Filtered.txt \
  --corrected-right-txt new.filters.txts/R.Filtered.txt \
  --corrected-sum-txt new.filters.txts/LR.Filtered.txt \
  --sum-mode vector_average
# Review the JSON and PASS metrics, then repeat the same command with --write.
git -C ../DRC/DRC-120.blue add -- \
  omdrc-designs/120.blue/rscreen-fdw8-20260813/design.json \
  "new.filters.txts/L 120.Rscreen.orig.txt" \
  "new.filters.txts/R 120.Rscreen.orig.txt" \
  new.filters.txts/LR.orig.txt \
  new.filters.txts/FLX.txt new.filters.txts/FRX.txt \
  FLX-trimmed-48k.wav FRX-trimmed-48k.wav \
  new.filters.txts/L.Filtered.txt new.filters.txts/R.Filtered.txt \
  new.filters.txts/LR.Filtered.txt
git -C ../DRC/DRC-120.blue commit -m "Declare 120.blue Rscreen FDW8 filter"
git -C ../DRC/DRC-120.blue tag -a 120.blue-rscreen-fdw8-20260813 \
  -m "120.blue Rscreen FDW8 correction"
```

Run the declaration command without `--write` first. For these exact current files it detects an 8192-sample causal delay and a nearly identical fixed gain of about -3.0003 dB in both WAVs relative to the FLX/FRX TXT responses. Only those two fixed transformations are fitted; the remaining complex-response error must pass the strict limits. The output path is not user-selectable: `omdrc-designs/<geometry>/<design-id>/design.json`.

Then run the complete build from `open-media-drc`, first without `--write`:

```
python3 scripts/new_filter_design.py \
  --source-root ../DRC/DRC-120.blue \
  --source-ref 120.blue-rscreen-fdw8-20260813 \
  --declaration omdrc-designs/120.blue/rscreen-fdw8-20260813/design.json
```

```
# Review every PASS line, then repeat with --write.
python3 scripts/verify_filter_bundle.py --all --require-sources
```

An annotated tag is mandatory by default. A raw commit requires the explicit lower-assurance `--allow-commit-ref` option. Without `--write`, the tool performs the complete build and changes no repository files. With `--write`, it uses this transaction:

1. Resolve every input to a regular file inside the source repository. Reject symlinks, duplicate roles, missing files, and modifications to selected tracked files. Require HEAD to equal the annotated tag's target; record and verify the tag object, source commit, declaration Git blob, and SHA-256s. Unrelated untracked files need not block deployment.
2. Parse the exported TXT headers. Require the declared L/R roles and room/setup markers, a common acoustic timing reference, and the same frequency grid. Smoothing is recorded and exposed in the UI rather than hidden.
3. Check filter TXT against filter WAV in complex frequency space after automatically detecting one integer causal delay and one constant export gain. Store both transformations, thresholds and residual error metrics in the manifest; reject frequency-dependent discrepancies.
4. Build everything in a new staging directory: copy canonical sources, generate every explicitly requested rate, calculate headroom, create graph data, and render the candidate manifest. Do not discover rates only from directories that happen to exist.
5. Re-read and hash every staged artifact. Parse each candidate BruteFIR config and require its rate, format, two coefficient paths, and attenuation to match the manifest. Require configured attenuation to be at least the calculated requirement, which already includes the chosen safety margin.
6. Calculate the response by complex multiplication. If explicitly declared, compare independent corrected L, R and/or L+R exports and reject differences beyond the declared RMS magnitude/phase limits.
7. Publish the complete staged bundle only after all mandatory checks pass. Never update half of an L/R pair or one rate at a time. Print the files Git should add and the new `bundle_id`.
8. Run `scripts/verify_filter_bundle.py --all` in CI and before installation. It verifies hashes, manifests, config mappings, analysis dependencies, and headroom without modifying files. A future CI rebuild job can run the dry-run builder in a clean checkout and demand byte identity for every RAW.

CLI and GUI responsibility

The many input switches are a one-time semantic role assignment, not routine deployment settings. Keep the CLI as the authoritative, scriptable backend and add a thin Qt Widgets front end for this selection step. A GUI does make this part safer because it can put an explanation, parsed REW title and validation result beside each `QPushButton` file picker. It must not implement separate DSP or deployment logic.

The proposed window has these rows:

1. source Git repository;
2. original left measurement;
3. original right measurement;
4. original L/R aggregate;
5. left filter response TXT and impulse WAV;
6. right filter response TXT and impulse WAV;
7. optional independently calculated corrected left, right and aggregate;
8. aggregate convention: coherent sum, vector average, or independent trace;
9. geometry, immutable design ID and description.

Each selected TXT row should immediately display the REW `Measurement`, `Source`, `Format`, `Dated`, `Note`, smoothing, row count and SHA-256. The GUI should reject duplicate paths, L/R grid or timing-reference differences, a filter TXT/WAV residual failure, and an aggregate convention that contradicts an unambiguous header such as `Source: Vector average`. It should show the detected WAV encoding, sample rate, delay and constant gain rather than asking the user to type them.

There should be no arbitrary output filename or deployment-folder picker. The read-only preview is derived from validated identifiers:

```
# In the source repository
omdrc-designs/<geometry>/<design-id>/design.json

# In open-media-drc
```

```
filters/<geometry>/source/<design-id>/measurement-L.txt
filters/<geometry>/source/<design-id>/measurement-R.txt
filters/<geometry>/source/<design-id>/measurement-L+R.txt
filters/<geometry>/source/<design-id>/filter-L.txt
filters/<geometry>/source/<design-id>/filter-R.txt
filters/<geometry>/source/<design-id>/filter-L.wav
filters/<geometry>/source/<design-id>/filter-R.wav
filters/<geometry>/analysis/<design-id>.json
filters/<geometry>/provenance/<design-id>.json
filters/<geometry>/<rate>/@<design-id>/L.raw
filters/<geometry>/<rate>/@<design-id>/R.raw
configs/<geometry>/brutefir-<rate>@<design-id>.conf.in
```

Forcing canonical *output* names is useful. Forcing an input picker to accept only a file named `L.txt` is not evidence that it contains the left measurement, and could encourage unsafe renaming. The semantic role, parsed metadata, content hash, declaration commit and annotated tag provide the identity. The GUI may offer `L.txt` as the suggested REW export name for future work, but it must still validate the content.

The safe GUI flow is `Review declaration` (always dry-run), `Write declaration`, then show the exact Git add/commit/annotated-tag commands. A separate `Deploy tagged design` action stays disabled until the checkout is clean, the tag is annotated and every declared hash is present at that tag. Qt 6 Widgets and PySide6 are available on the current development machine, so a small PySide6 front end can call this Python backend without a second language or DSP implementation.

`headroom_calc.py` is also a reusable CLI accepting a filter root, variant, format and safety margin. The obsolete list of root-level filenames is gone.

The intended two-repository commit order is:

1. Generate and review the source declaration, which explicitly assigns the measurement/filter roles. Commit it with the selected exports and WAVs.
2. Create an annotated tag at that commit (prefer a signed annotated tag).
3. Run deployment from that tag. The script verifies and records both the tag object and commit, then copies the selected sources.
4. Commit the manifest, sources, analysis, every rate pair, and config changes together in `open-media-drc`.
5. Optionally tag the `open-media-drc` commit with the geometry and short bundle ID as the deployment-side release anchor.

Offline response calculation

Prefer unsmoothed, time-aligned L/R measurement impulse WAVs as calculation inputs. Their FFTs and a time-domain convolution provide the cleanest check. If only REW frequency-response TXTs are available, reconstruct complex values from SPL and phase and interpolate the unwrapped complex response onto one common frequency grid.

For complex measurement responses `M_L`, `M_R` and delay-compensated filter responses `F_L`, `F_R`:

```
corrected-L   = M_L * F_L
corrected-R   = M_R * F_R
original-sum  = M_L + M_R
corrected-sum = (M_L * F_L) + (M_R * F_R)
```

L+R is a coherent complex sum, not an average of dB values. It is valid only when both measurements share the same acoustic timing reference. The analysis metadata must state whether the sum is raw (`L + R`) or divided by two for level normalisation; use the same convention before and after correction.

The audible runtime transfer also includes BruteFIR coefficient attenuation:

```
F_runtime = F_raw * 10 ** (-attenuation_dB / 20)
```

Store both the raw designed correction and the effective runtime correction. Default graphs should show the effective result, because the page promises the effect of the filter in use. A clearly labelled level-normalised view may be offered for comparing response shape.

The analysis file should contain a logarithmically reduced display grid, not only rendered images. Each dataset records the hashes of all inputs used to create it. Suggested traces are:

- original L, R, and coherent L+R;
- raw/effective FLX and FRX;
- corrected L and R;
- coherent corrected L+R.

Runtime and web UI binding

The existing `omdrcctrl` implementation already locates the running BruteFIR process, parses its exact `.conf`, reads the coefficient paths, and exposes a **Filter response** button. Extend that path rather than adding a second, geometry-selected source of truth.

For each request:

1. Read the `.conf` used by the running BruteFIR process.
2. Hash the exact L/R RAW files named by that config while reading them. Cache by path, inode, size, and high-resolution modification time only as a performance optimisation.
3. Find a manifest containing that exact `(relative path, SHA-256, rate, format)` pair. Verify the analysis-file hash and its recorded input hashes.
4. Check the config attenuation against the manifest. Compute the displayed active filter curve from the bytes just read, applying that attenuation.
5. Return graph data with a verification object and bundle details. Never label a response verified merely because `geometry` and `rate` match.
6. Put the provenance constraint in the always-visible green banner: annotated tag name, tag-object SHA, and exact source commit. The expandable details also show the declaration SHA-256, bundle ID, and active L/R RAW hashes.

Use two visible runtime states:

- **Verified** (green): active RAW hashes, config, analysis and source binding all pass. If independent corrected exports were declared, their numerical cross-check must also pass;
- **Mismatch / unverified** (red): no manifest matches the active bytes, a hash differs, the config differs, or analysis dependencies fail. The page may show a live FFT of the active RAW as diagnostics, but must not show stored room measurements as if they belonged to it.

The control page applies the same rule to A/B switching. During a switch it shows the previous and requested selectors. After `drc.sh` returns, the server re-reads the BruteFIR process, parses the config actually in use, hashes its L/R RAWs and reports the new selector as verified only if that runtime identity matches one manifest. A new immutable `@design-id` that starts but fails this check is an assurance failure, not a successful green switch. Legacy variants may still run, but remain visibly unverified.

Session persistence has one source of truth: `drc.sh`'s `last_geometry`, `last_arg` (mode/rate plus design selector), and `last_power`. Successful switches update those files automatically; the browser does not maintain a second copy. `drc.sh session` exposes their effective restore tuple as stable `key=value` output. The control page shows both the exact active identity and the auto-saved tuple, enables **Restore saved** only when they differ, and post-verifies the active config after restoration before claiming success. Restore deliberately honours a saved `off` state.

The page can use one Chart.js canvas with a **Magnitude / Phase** toggle and a custom checkbox legend shared by both modes. Default to only:

- independently measured Original L+R
- Corrected L+R

The coherent sum calculated from L/R, other channel, filter, and corrected traces remain one tap away. Group delay can be an optional advanced mode. A details panel should show the verification badge, bundle ID, active config/rate/attenuation, short L/R hashes, source Git commit, annotated tag object, human design description, declaration hash, optional `.mdat` archive hash, exported measurement headers/notes, smoothing and timing reference, prediction cross-check errors and headroom.

Installation requirement

`cmake/core-drc.cmake` installs each runtime manifest and analysis JSON beside the RAWs, while deliberately excluding `rew/`, `source/`, and source recipes. Source declarations, optional project archives and full exports remain development-only because their exact hashes and parsed metadata are embedded in the installed manifest.

CMake runs the read-only verifier before copying a geometry. It also rejects every new `@design` config that lacks its same-named provenance manifest. A broken bundle therefore fails configuration/packaging rather than producing a UI that looks authoritative; legacy variants without manifests remain installed only for compatibility and are visibly unverified in the response page.